Ex. No: 01

Image Handling and Pixel Transformations Using OpenCV

Name : DHARSHINI K **Reg. No :** 212225240034 ** Slot No :** 19AI406

In [2]:
```
# Import libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

1. Read the image ('Eagle_in_Flight.jpg') using OpenCV imread() as a grayscale image.

In [3]:
```
img = cv2.imread('Eagle_in_Flight.jpg', cv2.IMREAD_COLOR)
```

2. Print the image width, height & Channel

In [4]:
```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
h, w = img_rgb.shape[:2]
print(h, w)
```

```
600 768
```

3. Display the image using matplotlib imshow().

In [5]:
```
plt.imshow(img_rgb, cmap='viridis') # You can change 'viridis' to another cmap
plt.title("Original Image")
plt.axis('off') # Removes axis ticks and labels
plt.show()
```

4. Save the image as a PNG file using OpenCV imwrite().

In [6]:
```
img=cv2.imread('Eagle_in_Flight.jpg')
cv2.imwrite('Eagle.png',img)
```

Out[6]:
```
True
```

5. Read the saved image above as a color image using cv2.cvtColor().

In [7]:
```
img=cv2.imread('Eagle_in_Flight.jpg')
cv2.imwrite('Eagle.png',img)
```

Out[7]:
```
True
```

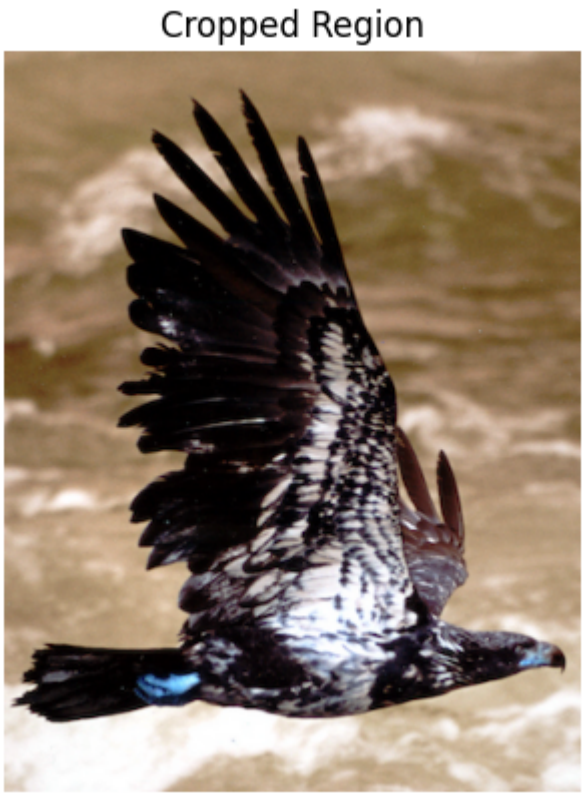
6. Display the Colour image using matplotlib imshow() & Print the image width, height & channel

In [8]:
```
plt.imshow(img)
plt.show()
img.shape
```

Out[8]: (600, 768, 3)

7. Crop the image to extract any specific (Eagle alone) object from the image.

In [9]:
```
crop = img_rgb[0:450,200:550]
plt.imshow(crop[:,:,::-1])
plt.title("Cropped Region")
plt.axis("off")
plt.show()
crop.shape
```



Out[9]:
```
(450, 350, 3)
```

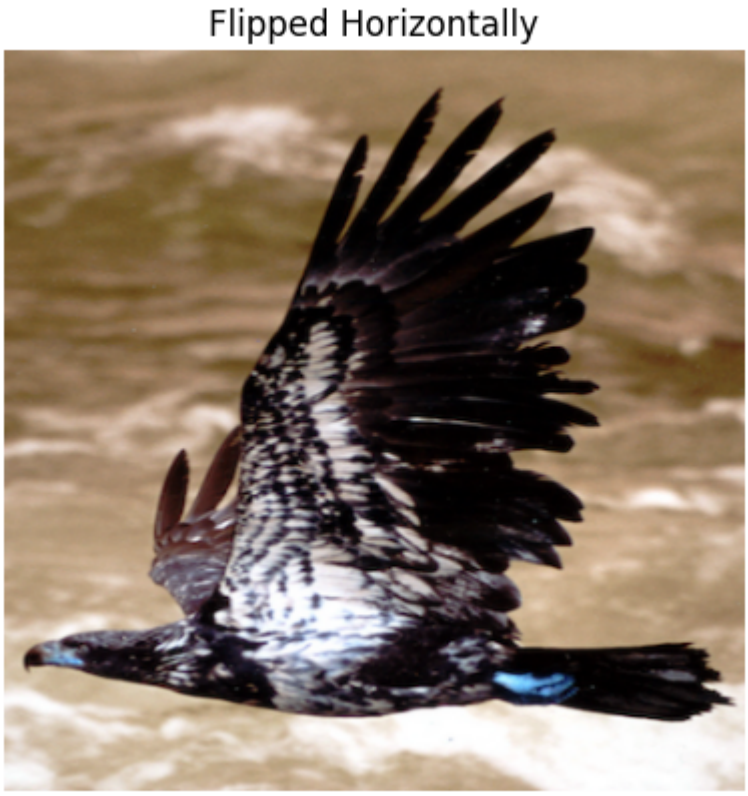
8. Resize the image up by a factor of 2x.

In [10]:
```
res= cv2.resize(crop,(200*2, 200*2))
```

9. Flip the cropped/resized image horizontally. (Eagle Image)

In [11]:
```
flip= cv2.flip(res,1)
plt.imshow(flip[:,:,::-1])
plt.title("Flipped Horizontally")
plt.axis("off")
```

Out[11]:
```
(np.float64(-0.5), np.float64(399.5), np.float64(399.5), np.float64(-0.5))
```



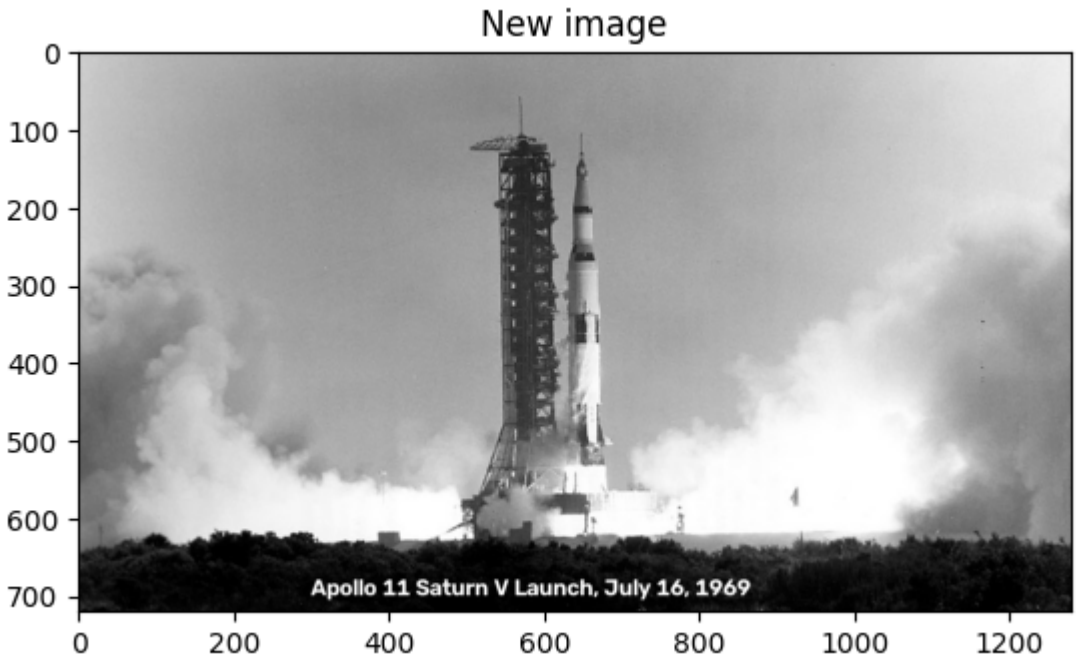
10. Read in the image ('Apollo-11-launch.jpg').

In [15]:
```
img=cv2.imread('Apollo-11-launch.jpg',cv2.IMREAD_COLOR)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img_rgb.shape
```

Out[15]:
```
(720, 1280, 3)
```

11. Add the following text to the dark area at the bottom of the image (centered on the image):

In [16]:
```
text = cv2.putText(img_rgb, "Apollo 11 Saturn V Launch, July 16, 1969", (300,
plt.imshow(text, cmap='gray')
plt.title("New image")
plt.show()
```



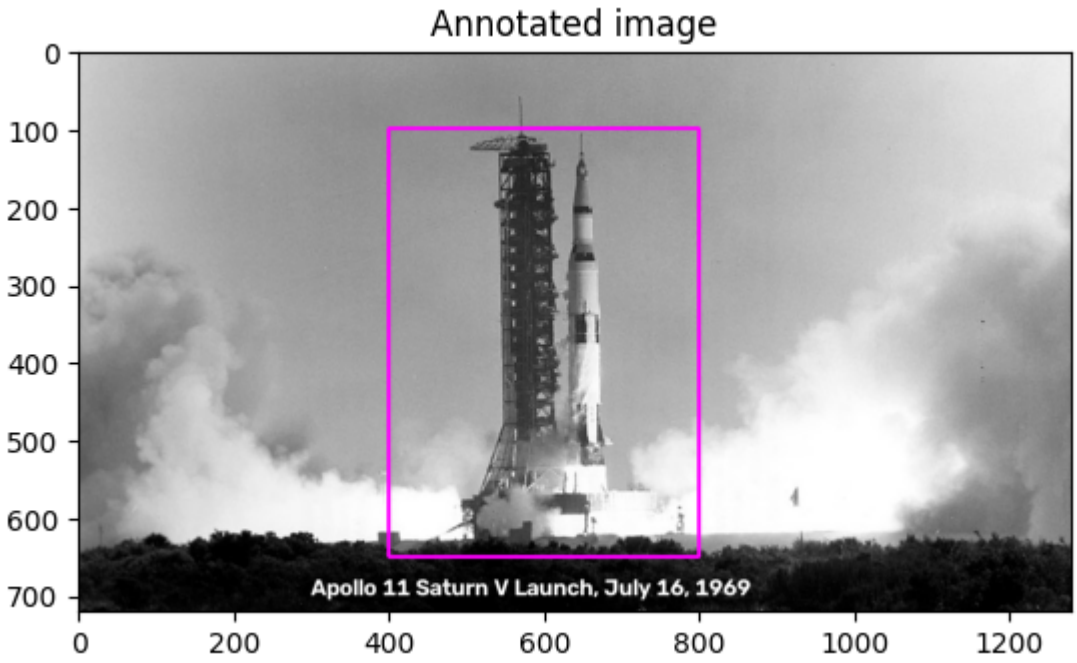
12. Draw a magenta rectangle that encompasses the launch tower and the rocket.

In [17]:
```
rcol= (255, 0, 255)
cv2.rectangle(img_rgb, (400, 100), (800, 650), rcol, 3)
```

Out[17]:
```
array([[[213, 213, 213],
        [213, 213, 213],
        [210, 210, 210],
        ...,
        [241, 241, 241],
        [240, 240, 240],
        [241, 241, 241]],

       [[213, 213, 213],
        [212, 212, 212],
        [209, 209, 209],
        ...,
        [240, 240, 240],
        [239, 239, 239],
        [240, 240, 240]],

       [[213, 213, 213],
        [212, 212, 212],
        [209, 209, 209],
        ...,
        [240, 240, 240],
        [240, 240, 240],
        [240, 240, 240]],

       ...,

       [[ 47,  47,  47],
        [ 46,  46,  46],
        [ 42,  42,  42],
        ...,
        [ 17,  17,  17],
        [ 11,  11,  11],
        [  5,   5,   5]],

       [[ 44,  44,  44],
        [ 47,  47,  47],
        [ 47,  47,  47],
        ...,
        [ 17,  17,  17],
        [ 10,  10,  10],
        [  4,   4,   4]],

       [[ 43,  43,  43],
        [ 49,  49,  49],
        [ 52,  52,  52],
        ...,
        [ 13,  13,  13],
        [  9,   9,   9],
        [  4,   4,   4]]], shape=(720, 1280, 3), dtype=uint8)
```

13. Display the final annotated image.

In [18]:
```
plt.title("Annotated image")
plt.imshow(img_rgb)
plt.show()
```



14. Read the image ('boy.jpg').

In [19]:
```
img =cv2.imread('boy.jpg',cv2.IMREAD_COLOR)
img_rgb= cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

15. Adjust the brightness of the image.

In [20]:
```
m = np.ones(img_rgb.shape, dtype="uint8") * 50
```

16. Create brighter and darker images.

In [21]:
```
img_brighter = cv2.add(img_rgb, m)
img_darker = cv2.subtract(img_rgb, m)
```

17. Display the images (Original Image, Darker Image, Brighter Image).

In [22]:
```
plt.figure(figsize=(10,5))
plt.subplot(1,3,1), plt.imshow(img_rgb), plt.title("Original Image"), plt.axis
plt.subplot(1,3,2), plt.imshow(img_brighter), plt.title("Brighter Image"), plt
plt.subplot(1,3,3), plt.imshow(img_darker), plt.title("Darker Image"), plt.axi
plt.show()
```

18. Modify the image contrast.

In [23]:
```
matrix1 = np.ones(img_rgb.shape, dtype="float32") * 1.1
matrix2 = np.ones(img_rgb.shape, dtype="float32") * 1.2
img_higher1 = cv2.multiply(img.astype("float32"), matrix1).clip(0,255).astype(
img_higher2 = cv2.multiply(img.astype("float32"), matrix2).clip(0,255).astype(
```

19. Display the images (Original, Lower Contrast, Higher Contrast).

In [24]:
```
plt.figure(figsize=(10,5))
plt.subplot(1,3,1), plt.imshow(img), plt.title("Original Image"), plt.axis("of
plt.subplot(1,3,2), plt.imshow(img_higher1), plt.title("Higher Contrast (1.1x)
plt.subplot(1,3,3), plt.imshow(img_higher2), plt.title("Higher Contrast (1.2x)
plt.show()
```

20. Split the image (boy.jpg) into the B,G,R components & Display the channels.

In [25]:
```
b, g, r = cv2.split(img)
plt.figure(figsize=(10,5))
plt.subplot(1,3,1), plt.imshow(b, cmap='gray'), plt.title("Blue Channel"), plt
plt.subplot(1,3,2), plt.imshow(g, cmap='gray'), plt.title("Green Channel"), pl
plt.subplot(1,3,3), plt.imshow(r, cmap='gray'), plt.title("Red Channel"), plt.
plt.show()
```

21. _merged the R, G, B , display along with orginal image

In [26]:
```
merged_rgb = cv2.merge([r, g, b])
plt.figure(figsize=(5,5))
plt.imshow(merged_rgb)
plt.title("Merged RGB Image")
plt.axis("off")
plt.show()
```

22. Split the image into the H, S, V components & Display the channels.

In [27]:
```
hsv_img = cv2.cvtColor(img, cv2.COLOR_RGB2HSV)
h, s, v = cv2.split(hsv_img)
plt.figure(figsize=(10,5))
plt.subplot(1,3,1), plt.imshow(h, cmap='gray'), plt.title("Hue Channel"), plt.
plt.subplot(1,3,2), plt.imshow(s, cmap='gray'), plt.title("Saturation Channel"
plt.subplot(1,3,3), plt.imshow(v, cmap='gray'), plt.title("Value Channel"), pl
plt.show()
```

23. _merged the H, S, V, display along with orginal image

In [28]:
```
merged_hsv = cv2.cvtColor(cv2.merge([h, s, v]), cv2.COLOR_HSV2RGB)
combined = np.concatenate((img_rgb, merged_hsv), axis=1)
plt.figure(figsize=(10, 5))
plt.imshow(combined)
plt.title("Original Image  &  Merged HSV Image")
plt.axis("off")
plt.show()
```

In [ ]: